



Planificación del Proyecto - Scrum

Temporis:

Una herramienta
accesible para el
control y la
optimización de
nuestro tiempo

1. Introducción y Metodología

Para el desarrollo de **Temporis**, se adopta el marco de trabajo ágil **Scrum**, adaptando estratégicamente sus eventos, ceremonias y artefactos a un entorno de **desarrollo individual (Solo-Scrum)**. El ciclo de vida del proyecto se divide de forma estricta en **Sprints fijos de dos semanas de duración**.

Esta ventana temporal garantiza un ritmo de inspección y adaptación constante, permitiendo desplegar **incrementos de software potencialmente entregables** tras cada ciclo. La flexibilidad de este enfoque facilita la integración inmediata de las pautas de accesibilidad y las correcciones de diseño identificadas durante las fases de prueba continua, mitigando así posibles riesgos de desviación, en el alcance final.

2. Product Backlog (Pila del Producto)

El *Product Backlog* se compone de historias de usuario y requerimientos técnicos desglosados y ponderados según su valor para el **Mínimo Producto Viable (MVP)** y su impacto en la **inclusión digital**. La priorización sigue una taxonomía estricta de dependencias técnicas:

- **Prioridad Alta (Bloque de Infraestructura y Core CRUD):**
 - Aprovisionamiento del entorno de desarrollo y SDKs esenciales.
 - Capa de persistencia *cloud* y aislamiento criptográfico de datos (*Firebase*).
 - Gestión de la lógica operacional y transaccional del temporizador.
- **Prioridad Media (Lógica del Contador, Arquitectura y UI Base):**
 - Sincronización asíncrona en segundo plano (Corrutinas de *Kotlin* y Servicios nativos).
 - Estructura arquitectónica limpia siguiendo el patrón MVVM.
 - Implementación de la interfaz responsiva y soporte nativo para el Modo Oscuro.
- **Prioridad Baja (Análítica Avanzada y Optimización de la Accesibilidad):**
 - Módulo estadístico y renderizado del mapa de calor de productividad (*heatmap*).
 - Temas de interfaz adaptados a entornos de baja visión (Modo de Alto Contraste).
 - Pulido fino de microinteracciones, transiciones visuales y animaciones inclusivas.

3. Planificación de *Sprints* (*Sprint Backlog*)

Sprint 0: Inicialización de la Infraestructura Base y Modelo de Datos *Cloud*

- **Duración:** “2 semanas.” [desarrollado aprovechando momentos libres durante un periodo de unos dos meses y medio. Resulta complicado definir una duración concreta para esta primera fase del desarrollo debido a que, en ese momento, el tiempo libre disponible era bastante limitado, compaginando el trabajo, el desarrollo de las tareas (*relacionadas con las otras asignaturas del CFGS en DAM*) y el desarrollo del proyecto en cuestión]
- **Tareas Programadas y Desglose Granular:**
 1. Configuración inicial del proyecto mediante el uso del *IDE* Android Studio, de *Gradle* (como herramienta de Gestión de dependencias) y también el control de versiones *Git*, a través de *Github*.
 2. Vinculación y autenticación del proyecto con la consola de *Firebase*.
 3. Implementación inicial de las *Security Rules* de *Cloud Firestore*, para el aislamiento de los datos, filtrando por el *UID* de cada usuario.
 4. Diseño y codificación de las clases de datos base (*POJOs* (*Plain Old Java Object*)/*Data Classes*) en *Kotlin*: *User.java*, *Timer.java*, *TimeRegister.java* y *Post.kt*.
 5. Integración del SDK de *Firebase Auth* y maquetación inicial de las pantallas de: *Login*, *Register*, *Timers*, *Timer* y “*User Dashboard*”.
- **Estado de Entrega (Tareas Realizadas):** “100%” completado y verificado en entorno local, aunque con ciertos errores, sobretodo relacionados con la actualización de los datos del usuario con la sesión iniciada, tratando de sincronizar e insertar/actualizar los nuevos datos, en *Firebase Firestore*.
- **Comentario acerca del “*sprint* inicial”:** Esta primera fase del desarrollo del proyecto se realizó durante el periodo de tiempo aproximado que comprendió desde mediados de Febrero hasta mediados de Mayo (ambos del año 2025), durante el desarrollo de la propuesta inicial de aplicación para Android. Se entregó en tiempo y forma adecuados, aunque manifestando y “manteniendo” ciertos errores/disfunciones, de forma continuada.

Sprint 1: Infraestructura Base y Modelo de Datos *Cloud*

- **Duración:** 2 semanas.
- **Tareas Programadas:**
 1. Ajuste y mejora de la configuración del proyecto mediante el uso del *IDE* Android Studio, de *Gradle* (como herramienta de Gestión de dependencias) y también el control de versiones *Git*, a través de *Github*. En esta primera etapa se tratan de solucionar o mejorar los errores relacionados principalmente con la sincronización/actualización de los datos del usuario, al tratar de actualizarlos y guardarlos en *Firebase Firestore*.
 2. Vinculación y autenticación del proyecto con la consola de *Firebase*.
 3. Implementación inicial de las *Security Rules* de *Cloud Firestore*, para el aislamiento de los datos, filtrando por el *UID* de cada usuario.
 4. Diseño y codificación de las clases de datos base (POJOs (*Plain Old Java Object*)/*Data Classes*) en *Kotlin*: *User.java*, *Timer.java*, *TimeRegister.java* y *Post.kt*.
 5. Integración del SDK de *Firebase Auth* y maquetación inicial de la pantalla de *Login* tradicional.
- **Estado de Entrega (Tareas Realizadas):** “100%” completado y verificado en entorno local, aunque con ciertos errores.

Sprint 2: Arquitectura MVVM y Lógica de Negocio (*Core Engine*)

- **Duración:** 2 semanas.
- **Tareas Programadas:**
 1. Creación de la estructura de paquetes según *Clean Architecture* (*adapter*, *model*, *repository*, *ui* y *util*).
 2. Implementación del patrón *Repository* (*TimerRepository.kt*, *UserRepository.kt* y *PostRepository.kt*) para la abstracción de operaciones CRUD con *Firestore*.
 3. Desarrollo de los *ViewModels* (*TimerViewModel.kt*) con control de estado mediante *MutableStateFlow*.
 4. Despliegue del motor de cuenta atrás del temporizador mediante hilos asíncronos eficientes (*Kotlin Coroutines*).
- **Estado de Entrega (Tareas Realizadas):** Completado. Código integrado en la rama principal (*main*) tras superar los test unitarios.

Sprint 3: Diseño de Interfaces Responsivas y Accesibilidad Universal (LSE/WCAG)

- **Duración:** 2 semanas.
- **Tareas Programadas y Desglose Granular:**

1. Diseño y maquetación del árbol de vistas XML utilizando componentes adaptativos (`ConstraintLayout`).
 2. Implementación y unificación del sistema de diseño visual (estilos de botones redondeados, gradientes y jerarquía tipográfica).
 3. Creación e integración de los recursos de color para soporte nativo de **Modo Oscuro** y **Modo de Alto Contraste**.
 4. Inyección de atributos `contentDescription` en todos los componentes interactivos e imágenes para la correcta verbalización en el lector de pantallas *TalkBack*.
 5. Ajuste del escalado tipográfico adaptativo dinámico en unidades `sp` (hasta 30sp) previniendo el desbordamiento o colapso de las vistas de la interfaz.
- **Estado de Entrega (Tareas Realizadas):** Completado. Las vistas de configuración de accesibilidad y selección de modos son completamente funcionales.

Sprint 4: Control de Calidad (QA), Inspección Estática y Cierre

- **Duración:** 2 semanas.
- **Tareas Programadas y Desglose Granular:**
 1. Configuración del servidor local de **SonarQube** y ejecución del plugin de Gradle para análisis estático de código.
 2. Refactorización y limpieza de la complejidad ciclomática en ViewModels y erradicación de *code smells* detectados.
 3. Escritura y ejecución de baterías de pruebas unitarias con JUnit e instrumentales de interfaz con el framework Espresso.
 4. Auditoría de campo final de accesibilidad móvil con la herramienta de diagnóstico *Accessibility Scanner* de Google.
 5. Compilación del artefacto final corregido y redacción de las memorias técnicas y de control de calidad.
- **Estado de Entrega (Tareas “Realizadas”):** En proceso. El objetivo de la cobertura de código que queremos validar en un futuro, debería de encontrarse por encima del 85% y con la pasarela de calidad (*Quality Gate*) aprobada.

4. Tablero Kanban (Herramienta de Gestión)

Para el seguimiento visual, se ha utilizado un tablero **Kanban**, creado y administrado mediante **GitHub Projects**, en el cual, las tareas, han ido transitando/desplazándose por/a través de las columnas definidas: *New issues*, *Ice Box*, *Product Backlog*, *Sprint Backlog*, *In Progress*, *In Review* y *Done*. Esto me permite detectar cuellos de botella en la fase de diseño de interfaces.